1. **Introduction**

This document describes the design of the worker client in the RJDAP system. It explains how a worker registers with the server, receives computational tasks, processes them, sends results, and handles acknowledgements and retransmissions.

2. **Worker registration**
    - Worker starts
        The worker creates a UDP socket so it can communicate with the server.
    - Worker sends `WORKER_JOIN`
        It tells the server that a new worker wants to join.
    - Server registers the worker
        The server adds it to its worker list and assigns a unique Worker ID.
    - Server sends `WORKER_ACCEPT`
        The response contains the assigned Worker ID.
    - Worker stores its ID
        The worker is now registered and can receive tasks.

Registration Retry and Duplicate Registration

If a worker sends another `WORKER_JOIN` message from the same IP address and UDP port, the server returns the worker's previously assigned Worker ID instead of assigning a new one.

This allows the worker to retry registration if the original `WORKER_ACCEPT` response is lost, without creating duplicate worker entries on the server.

3. **Worker state machine**

The worker client uses a state machine to manage its behavior throughout its lifecycle. Each state represents a specific activity performed by the worker.

The proposed worker states are:

1. **STARTING:** The worker application starts and creates its UDP socket.
2. **REGISTERING:** The worker sends a `WORKER_JOIN` message and waits for a `WORKER_ACCEPT` response from the server.
3. **IDLE:** The worker has successfully registered and is waiting to receive a task.
4. **BUSY:** The worker has received a `TASK_DISPATCH` message and is executing the assigned computation.
5. **WAITING_ACK:** After completing the computation, the worker sends a `TASK_RESULT` message and waits for an `ACK_RESULT` message from the server.
6. **DISCONNECTED:** The worker enters this state when communication with the server fails after the agreed retry limit.

State Transitions:

The normal worker lifecycle is:

`STARTING → REGISTERING → IDLE → BUSY → WAITING_ACK → IDLE`

The worker returns to the `IDLE` state after receiving a valid acknowledgement, allowing it to receive another task.

If registration or result communication fails, the worker follows the agreed timeout and retry procedures. If communication cannot be restored, it may enter the `DISCONNECTED` state.

Heartbeat Mechanism

The worker uses a `HEARTBEAT` message to periodically inform the server that it is still active and communicating.

The heartbeat mechanism allows the server to monitor worker availability and detect possible communication failures.

- **Heartbeat Configuration:**
  Heartbeat Interval: 2 seconds.
  Heartbeat Timeout: 6 seconds.
- **Heartbeat Process:**

1. After successful registration, the worker periodically sends a `HEARTBEAT` message to the server every 2 seconds.
2. The server receives the heartbeat and updates the worker's last-seen time.

3. If the server does not receive a heartbeat from a worker for 6 seconds, it marks that worker as inactive.
4. An inactive worker should not receive new tasks until it becomes active again.

4. Task Reception and Computation

The worker is responsible for receiving computational tasks from the server and executing the assigned computation.

The proposed task contains three parameters:

- `start`: The beginning of the search range.
- `end`: The end of the search range.
- `target`: The number that the worker must search for.

Task Processing Steps

1. The worker waits in the `IDLE` state.
2. The server sends a `TASK_DISPATCH` message containing the task information.
3. The worker receives and decodes the message.
4. The worker extracts the `start`, `end`, and `target` values.
5. The worker searches for the target number within the assigned range.
6. If the target is found, the worker prepares a `SUCCESS` result containing the target number.
7. If the target is not found, the worker prepares a `NOT_FOUND` result.

After completing the computation, the worker prepares a `TASK_RESULT` message to send the result back to the server.

5. Result Transmission

After completing a computational task, the worker prepares and sends a `TASK_RESULT` message to the server.

The message contains the following information:

- **Task ID:** Identifies the task whose computation was completed.
- **Worker ID:** Identifies the worker that performed the task.
- **Attempt Number:** Identifies the current execution attempt.

- **Payload:** Contains the computation status and result.

The result payload has a total size of 5 bytes:

Field	Size
Status	1 byte
Result	4 bytes

The possible status values are:

- `SUCCESS (1)`: The target number was found.
- `NOT_FOUND (2)`: The target number was not found in the assigned range.
- `ERROR (3)`: An error occurred during computation.

The worker sends the `TASK_RESULT` message to the server using UDP. After sending the result, it enters the `WAITING_ACK` state and waits for confirmation from the server.

6. ACK Reception and Retransmission

Since RJDAP uses UDP, messages may be lost during transmission. Therefore, the worker implements an acknowledgement and retransmission mechanism to improve the reliability of result delivery.

ACK Reception and Validation

After sending a `TASK_RESULT` message, the worker waits for an `ACK_RESULT` message from the server.

When an acknowledgement arrives, the worker checks the following fields:

- **Task ID:** Must match the task whose result is awaiting acknowledgement.
- **Worker ID:** Must match the worker's assigned ID.
- **Attempt Number:** Must match the current task attempt.

If all fields match, the worker considers the result successfully acknowledged, clears the pending result, and returns to the `IDLE` state.

Retransmission Mechanism

If the worker does not receive a valid acknowledgement before the timeout expires, it retransmits the same `TASK_RESULT` message.

The proposed retransmission configuration is:

- **ACK Timeout:** 1 second.
- **Maximum Retransmissions:** 3.
- **Total Transmission Attempts:** 4 (one initial transmission and three retransmissions).

The worker must preserve the original result message during retransmission. It must not recompute the task or change its Task ID, Worker ID, Attempt Number, or result payload.

Behavior After Maximum Retransmissions

If the worker still does not receive a valid acknowledgement after three retransmissions:

1. The worker stops retransmitting the result.
2. It enters the `DISCONNECTED` state.
3. It preserves the pending result instead of discarding it.
4. It does not accept new tasks until communication recovery is handled.

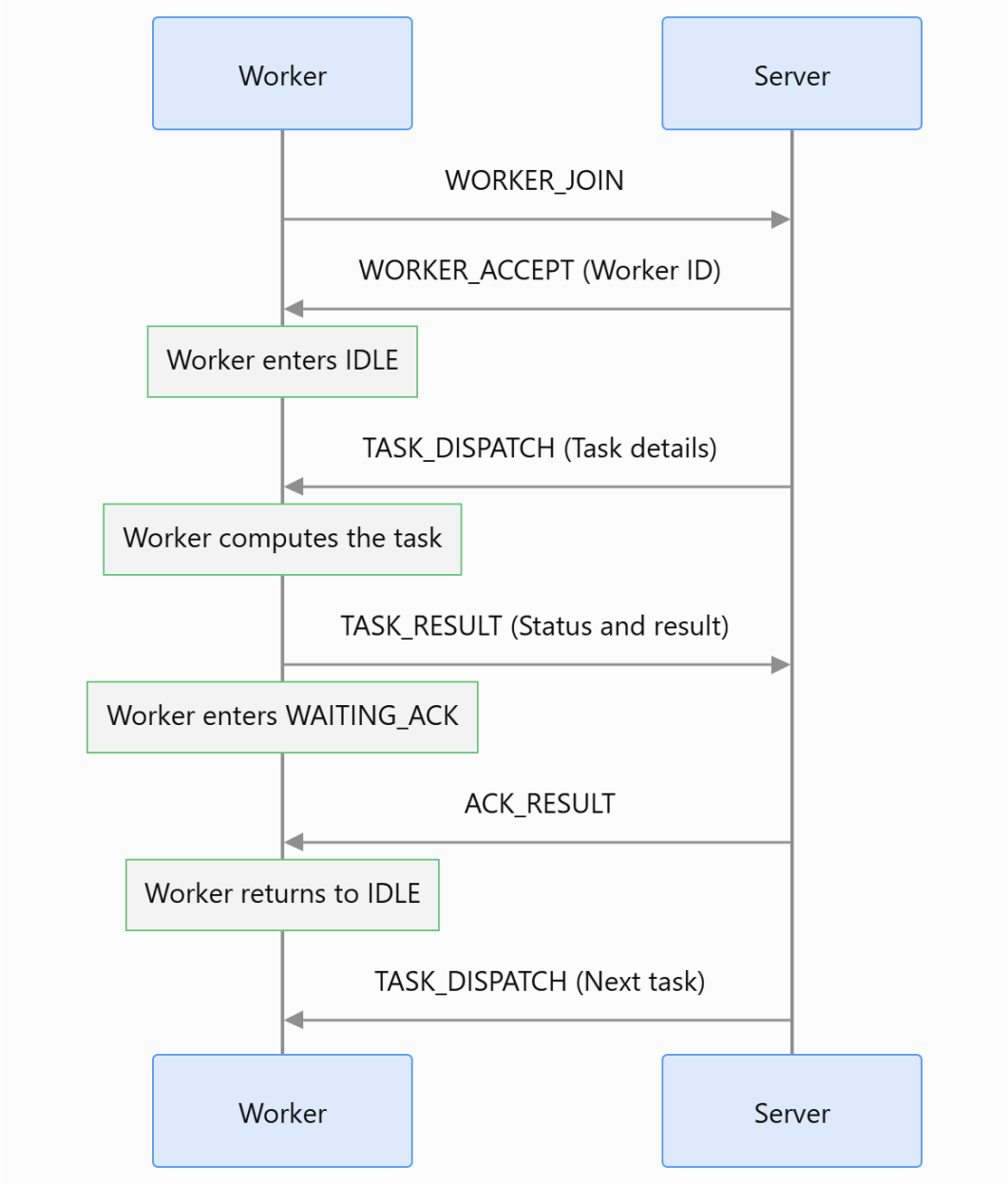
The pending result must remain available so that it can be handled by a future recovery mechanism.

7. Worker–Server Communication Sequence

This section describes the complete communication process between the worker and the central server.

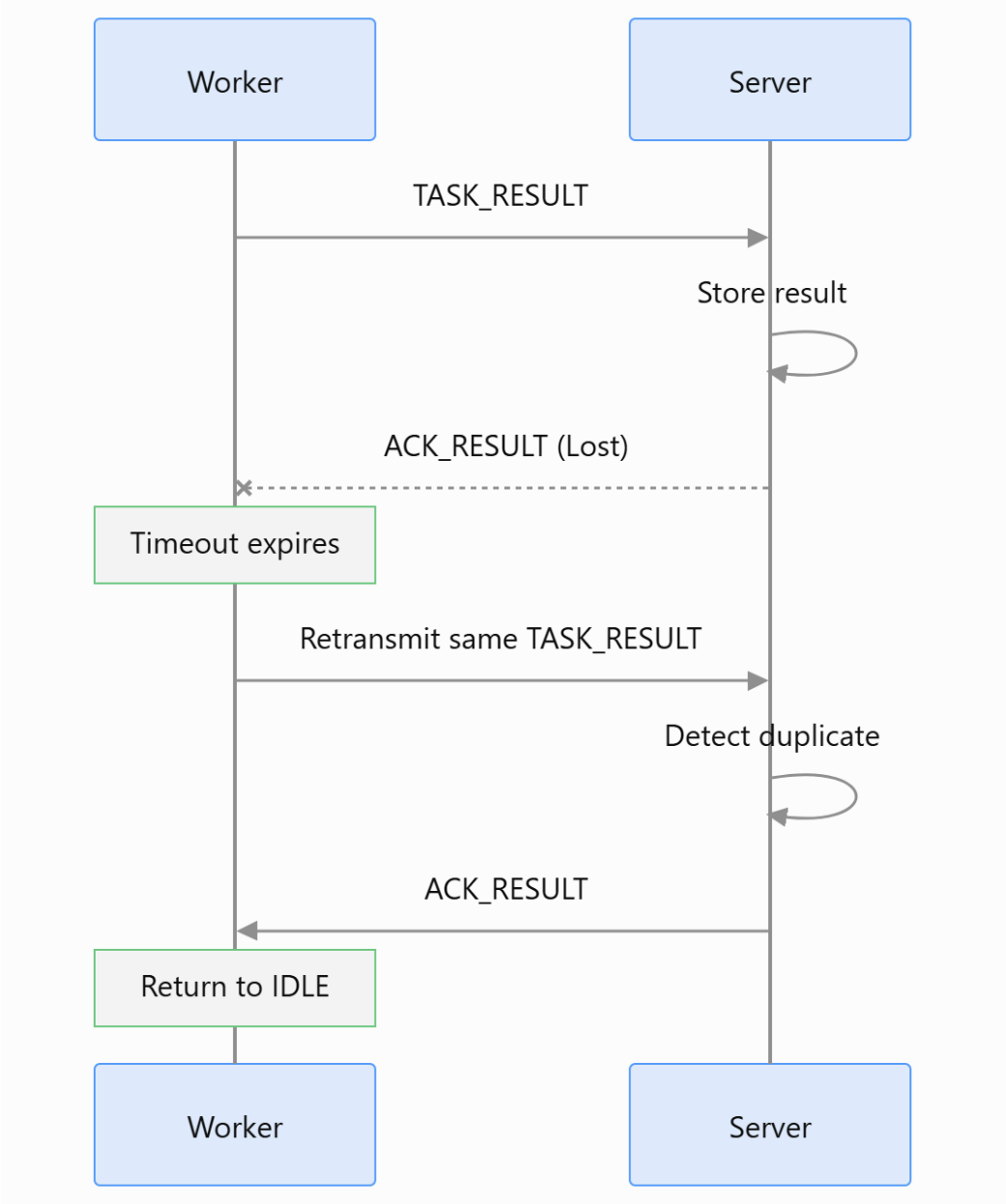
Normal Communication Sequence

The normal communication process consists of the following steps:



Communication Failure Scenario

If the server receives and stores a result but the acknowledgement is lost:



This process helps the system handle lost UDP messages without storing the same task result multiple times.